Project 4 - cpmFS
Project Report

COMP7500 - Advanced Operating Systems

Chris HInkson

I. Table of Contents

I. Table of Contents 2
II. Project Introduction 4
III. Report Checklist 5
IV. Submission Checklist 5
V. Architecture and Design 6
- **1. Data Flow Diagram** 6
- 2. Discussion on Design Issues 7
  - A. Accessing Extent Data 7
  - B. Powerhouse vs Utilities 7
- **3. System Modules** 10
- 4. Function Prototypes 11
  - A. fsysdriver Module 11
  - A. diskSimulator Module 13
  - B. cpmfsys Module 16

VI. Lessons Learned 23
- **1. Master of All Trades vs Jack of One** 23
- 2. Don't Make Assumptions with Someone Else's Code 23
- 3. Accept Not Having Control 23
- 4. Don't Be Afraid To Start Over 24

VII. Conclusion 25

II. Project Introduction

The goal of this project is to finish implementing the cpmFS, a simple file system. The main portion to be completed of this is the layer of abstraction between file names and extents / blocks.

III. Report Checklist

To help simplify the evaluation of this report, you can find sections for each specific criteria mentioned in the assignment instructions for this report here:

Required Criteria	See Sections
Design and Implementation of cpmFS	Architecture and Design
Lessons Learned	Lessons Learned

IV. Submission Checklist

To help simplify the evaluation of this project, you can find deliverables at the following locations:

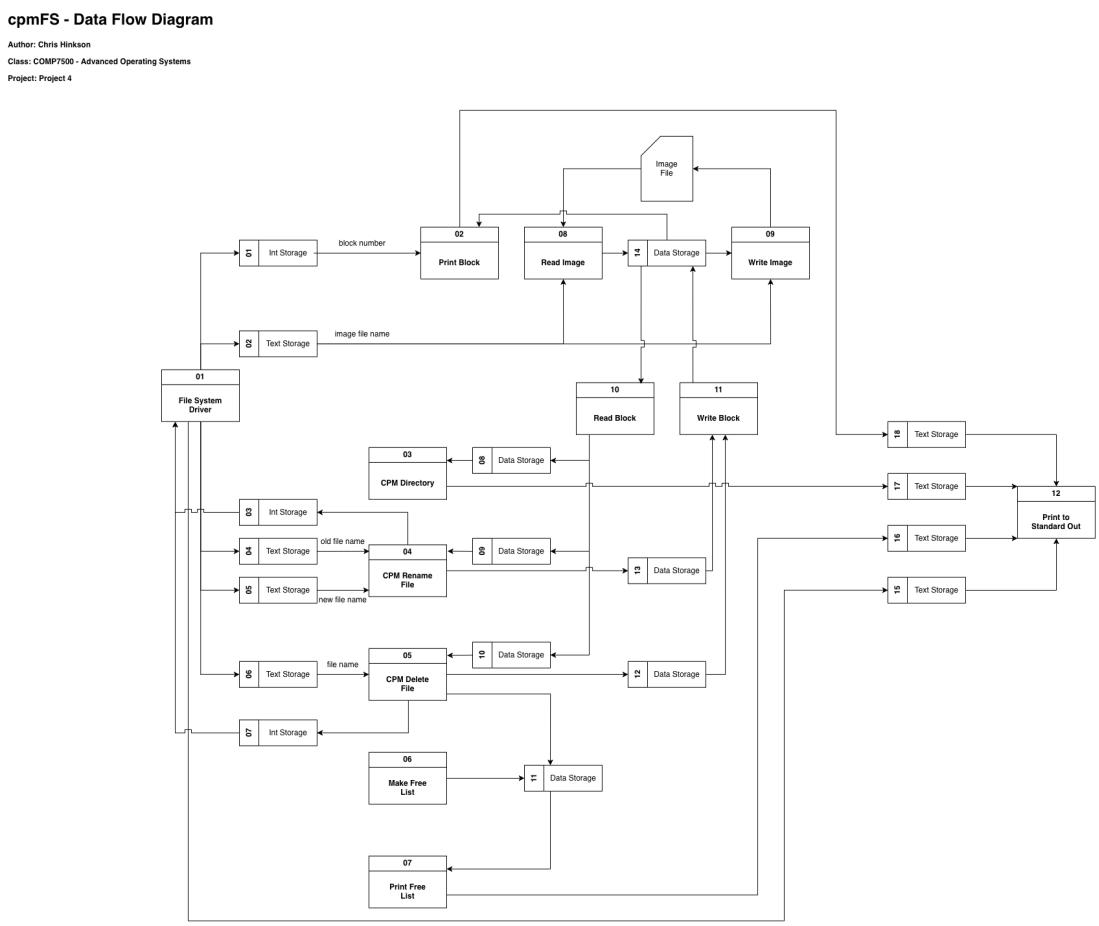
Required Criteria	Location
Report	Here 🙂
Source Code	“cpmfsys.c”

V. Architecture and Design

Below I discuss several aspects of the design for cpmFS. As discussed in the [Project 4 Specifications] and [Project 4 FAQ], even though the majority of design was pre-determined and most work completed was simple implementation, I do my best at thoroughly detailing and discussing the choices made here.

1. Data Flow Diagram

Below is my Data Flow Diagram. In the diagram, I represent various structures as data storages, and clarify what each actually is when possible.



2. Discussion on Design Issues

A. Accessing Extent Data

One key design, and also partially implementation, issue is that preference of data access for extents. I identified two main ways that this can be done: either direct access can occur from the initial directory reference or a DirStruct can be made and written each time data access is needed. Both methods had several pros and cons, and would lead to different design methodologies surrounding the cpm architecture. For instance, primarily using direct access across the cpm file system may simplify small operations, improve memory usage, and enhance the speed for various actions. On the other hand, requiring any data access to use DirStruct's would enforce common behaviors, introduce structure, and reduce the potential for errors to occur. For these reasons, I chose that enforcing the use of DirStruct's everywhere would be the best policy for my implementation. This choice pushed my design toward a more structured and multilayered architecture, rather than allowing random modifiers with no hierarchical relationships. I believe that this is the best design policy, as it allows for the project to be extended without troublesome maintenance or unreasonable refactors to occur. Additionally, based on the given assignment requirements, I believe that this was also the desired design methodology as well.

B. Powerhouse vs Utilities

For a simple implementation project such as this, one key consideration in my design was whether to take the form of "powerhouse" functions, which can do more on their own and can provide the bare skeleton for their operations, or to rather create "utilities", which provided common operations at the expense of overhead or sometimes unneeded additional operations. For instance, with the parsing of file names and application of naming rules, each instance in which a file name is handled could have been implemented with separate logic that was very specific to that single operation. While this may seem most opportune in each separate instance, from

a high level, this removes the idea of having a “master” of file names. Instead, I chose the approach to make a very strong design and function for anything dealing with file names, at the sacrifice of some overhead.

To show more about why this is important for the project design (since I feel my above description is rather abstract), I’d like to point out a key example in how the design was turned into implementation. For file name parsing, slightly different conditions or formats are needed for simply validating a file name vs comparing two file names. Due to the storage methods available, conditions could be things like which segments are found, and formats could be things like termination with and without padding. As a result, there are two direct routes: either each operation has to introduce its own name parsing logic as specific as possible, or a slightly more bulky but commonized parsing logic is provided for all scenarios. In line with my design choice, I chose the latter, and implemented both a new structure type to contain file parsing information and several utilities to work with this new structure type. In this way, I made all file name parsing and rule enforcement standardized, and simply extract the information needed for each operation from all of the information available after parsing.

C. Usability and Interactivity

While not one of my design issues, one that I think is always most important to look at is the usability and interactivity of the system. Looking at the current form of cpmFS, this is essentially only as an interface, and requires that a user program (such as the fsdriver) be provided to actually work with the interface. However, looking at both the requirements of the cpmFS and the diskSimulator modules, there is a mix of ending features (where data is output to some kind of channel rather than trickled back up) and returning features (where the output of operations is trickled back up). To me, this complicates things - each layer is partially on its own, but not quite. This feels like a mix of design to me. Instead, I would opt for a stricter multi-layer design, where neither cpmFS or diskSimulator offer ending user features. Any output should only be internal logs produced to some kind of system log file, and any desired user output should rather be data passed back up to the above layer. This would help to keep the

different modules contained, rather than allowing overreach or mixing.

If this kind of stricter design were to be implemented, then several changes would need to occur. First, no “debug” functions should exist - rather, they should simply produce either specialized structure types containing the desired data or simple data types such as strings or integers for smaller pieces of data. Nothing below the fsdriver should be allowed to produce normal user output (such as printing the free list or blocks). At the fsdriver, more user-focused features could be provided that interface with the lower levels. For instance, perhaps the cpm interface provides a way to get all of the blocks for a file name, or a way to get a shallow copy of the free list that can then be formatted in a variety of ways at the driver level. These changes would help to improve the design and enforce modular separation.

D. A New Layer

One of the key design choices I recognized in this project was the very “wide” or “spread out” cpmFS layer. In the single layer, it contains both lower-level utilities to modify the free list and individual extents, while also providing an interface to work with file names and the directory at a high level. To me, with how much is being performed in a singular layer, this could be better separated. Either side-by-side modules or hierarchical modules could help improve the design here. If side-by-side modules were used, perhaps one module could be made strictly over the free list, one module could be made over the directory and extents, and a final third one could serve utilities for file name parsing and rule validation. If hierarchical modules are used, perhaps the highest-layer provides a similar interface to the current (list, delete, rename), a middle layer allows for abstract operations on the directory and extents, and a lower layer allows for specific translations from these directory/extent operations into memory, block references, and the free list. Overall, I think either of these options would provide a more rigid structure for the cpmFS, and would have made the purpose of each part of the project more clear and separated from the rest.

3. System Modules

Below I provide a quick description of each system module.

Module: fsysdriver
Code File: src/fsysdriver.c **Header File:** none
Description The fsysdriver module is the main control flow for using cpmFS.

Module: diskSimulator
Code File: src/diskSimulator.c **Header File:** src/diskSimulator.h
Description The disk simulator module allows for using an image file as a mock storage disk. This allows us to avoid touching actual storage and use a virtual storage instead, making sharing created storages and performing modifications safely much easier.

Module: cpmfsys
Code File: src/cpmfsys.c **Header File:** src/cpmfsys.h
Description The cpmfsys module is the central module for providing the CPM file system. This includes a variety of utilities for working with files and the interfacing with the diskSimulator. It primarily includes initializations and abstractions for deleting and renaming files so that an end user can do so without touching the block level.

4. Function Prototypes

A. fsysdriver Module

Function: printBuffer
Prototype Signature **`void printBuffer(uint8_t buffer[], int size)`**
Description This function prints a region of memory for debugging purposes.
Parameters

Type	Name	Description
uint8_t[]	buffer	A buffer of uint8_t
int	size	The number of bytes to print off from the buffer

Return Value void

Function: main

Prototype Signature
`int main(int argc, char * argv[])`

Description
Main program entry.

Parameters

Type	Name	Description
int	argc	Count of arguments given
char*	argv	Arguments given

Return Value
int of 0 for success or other for error

A. diskSimulator Module

Function: blockRead

Prototype Signature
`int blockRead(uint8_t *buffer, uint8_t blockNum)`

Description
Read a block from the (virtual) disk.

Parameters

Type	Name	Description
uint8_t*	buffer	Buffer to read the block into
uint8_t	blockNum	Number of the block to read into the buffer

Return Value
int of 0 for success

Function: blockWrite

Prototype Signature
`int blockWrite(uint8_t *buffer, uint8_t blockNum)`

Description
Write a block onto the (virtual) disk.

Parameters

Type	Name	Description
uint8_t*	buffer	Buffer to be written as the block.
uint8_t	blockNum	Number of the block to write the buffer to.

Return Value
int of 0 for success

Function: printBlock

Prototype Signature
`void printBlock(uint8_t blockNum)`

Description
Print a block from the (virtual) disk.

Parameters

Type	Name	Description
uint8_t	blockNum	Number of the block to print out.

Return Value
void

Function: writeImage

Prototype Signature
`size_t writeImage(char *fileName)`

Description
Write the virtual disk to an image file.

Parameters

Type	Name	Description
char*	fileName	The file name of the image file to write the virtual disk to.

Return Value
The number of bytes that were written.

Function: readImage

Prototype Signature
`size_t readImage(char *fileName)`

Description
Read the virtual disk from an image file.

Parameters

Type	Name	Description
char*	fileName	The file name of the image file to read the virtual disk from.

Return Value
The number of bytes that were read.

B. cpmfsys Module

Function: mkDirStruct
Prototype Signature **`DirStructType *mkDirStruct(int index, uint8_t *e)`**
Description Create a DirStructType for the referenced extent from disk.
Parameters

Type	Name	Description
int	index	The index of the extent in the directory
uint8_t*	e	Pointer to disk block 0, which is the cpm directory

Return Value
DirStructType containing the status, name, extension, XL, BC, XH, RC, and blocks from the specified extent.

Function: writeDirStruct

Prototype Signature
`void writeDirStruct(DirStructType *d, int index, uint8_t *e)`

Description
Write information from a DirStructType to the referenced extent on disk.

Parameters

Type	Name	Description
DirStructType*	d	Pointer to the DirStructType object containing the information to be written
int	index	The index of the extent in the directory
uint8_t*	e	Pointer to disk block 0, which is the cpm directory

Return Value
void

Function: makeFreeList

Prototype Signature
`void makeFreeList()`

Description
Initializes the free list for the current disk.

Parameters

Type	Name	Description
none	none	none

Return Value
void

Function: printFreeList

Prototype Signature
`void printFreeList()`

Description
Prints the free list for the current disk.

Parameters

Type	Name	Description
none	none	none

Return Value
void

Function: cpmDir()

Prototype Signature
`void cpmDir()`

Description
Prints the directory for the current disk, including the name, extension, and size of all files in the directory.

Parameters

Type	Name	Description
none	none	none

Return Value
void

Function: checkLegalCharacter

Prototype Signature
`bool checkLegalCharacter(char c)`

Description
Checks if a singular character is legal or not for a file name. Legal characters are any alphanumeric character, a period ('.'), or a terminator byte ('\0'),

Parameters

Type	Name	Description
char	c	The character to check.

Return Value
Bool of true if legal or false if illegal.

Function: initializeNameInformationStruct

Prototype Signature
`void initializeNameInformationStruct(nameInformationStructType *nameInfo)`

Description
Initializes the referenced nameInformationStructType with default values. For booleans, this default value is false, and for integers, this default value is the macro CPM_INT_UNDEFINED. This macro is defined at the top of `cpmfsys.c`, and is currently set to a value of -7500 (any non-naturally-occuring value for integers used here, I simply used the course code).

Parameters

Type	Name	Description
nameInformationStructType(	nameInfo	Pointer to a nameInformationStructType to set all fields to default values.

Return Value
void

Function: getNameSegmentIndices

Prototype Signature
`nameInformationStructType getNameSegmentIndices(char *name)`

Description
Parses a given series of characters into a name information object and checks it against naming rules.

Parameters

Type	Name	Description
char*	name	Pointer to the first character of the target to parse.

Return Value
A nameInformationStructType that includes a boolean for if the name segment was found, a boolean for if the separator was found, a boolean for if the extension was found, a boolean for if the terminator was found, the start index of the name segment, the end index of the name segment, the index of the separator, the start index of the extension, the end index of the extension, and the index of the terminator.

Function: checkLegalName

Prototype Signature
`bool checkLegalName(char *name)`

Description
Checks whether a given file name is legal or not according to rules.

Parameters

Type	Name	Description
char*	name	Pointer to the first character of the target to check.

Return Value
A boolean of true if the name is legal or false if the name is illegal

Function: findExtentWithName

Prototype Signature
`int findExtentWithName(char *name, uint8_t *block0)`

Description
Finds the extent in the directory for a given filename.

Parameters

Type	Name	Description
char*	name	Pointer to the first character of the file name to find the extent for.
uint8_t*	block0	Pointer to block 0, the file directory.

Return Value
An int of (-1) if the name was illegal or not found in the directory, else (0 to 31) the index of the matched extent for the file name

Function: cpmDelete

Prototype Signature
`int cpmDelete(char *name)`

Description
Deletes an extent for the given filename from the file directory.

Parameters

Type	Name	Description
char*	name	Pointer to the first character of the file name to delete from the directory.

Return Value
An int of (0) for success or (-1) if the file name was illegal or not found in directory.

Function: cpmRename

Prototype Signature
`int cpmRename(char *oldName, char *newName)`

Description
Renames a file in the directory.

Parameters

Type	Name	Description
char*	oldName	Pointer to the first character of the file name from the directory to rename.
char*	newName	Pointer to the first character of the new name for the file to be renamed.

Return Value
An int of (0) for success or (-1) if the file name was illegal or not found in directory.

VI. Lessons Learned

Several lessons were learned during development of this project. I outline the major ones here.

1. Master of All Trades vs Jack of One

Connecting to a point I made in my design discussions, I think an important choice I made was to implement a central structure and utilities for working with file names. Although this does add some overhead and unnecessary background to file name operations, it allows a singular point for total control over all file names and rule enforcement. If each part of the project working with file names was allowed to carry its own logic and rule enforcement, then each may be *okay* at one specific part of file name logic, but none would truly be a *master* at file names and would create problems for extensibility. Picking up on this issue early on and making steps to fix it before continuing the project proved to save me much time later on in the implementation.

2. Don't Make Assumptions with Someone Else's Code

Although I have recognized my tendency to do this long before this project, early on in the project I began to make assumptions about the rest of the code (which was not written by me) that were, quite frankly, wrong. While I can read code well for the most part, it is still easy to make design assumptions when you are not totally familiar with the ins and outs of a codebase. Additionally, with some of the provided comments, not all functionality in the rest of the codebase was clear; why are some functions designed this way, why does it want this input, etc. Due to the nature of this being a project, much of this was eventually revealed, but I think not making assumptions early on would have been helpful. If I had rather spent more time studying the codebase and considering how each part was made at the lowest level, then I may have been able to align my design and implementation more closely from the beginning.

3. Accept Not Having Control

Somewhat specific to this project, one part that I found rather troublesome (I've slightly touched on this above) is the lack of control over other modules. To stay inline with project requirements, I only worked out of cpmfs - however, I quite often wanted to be able to change or at least discuss the design of other portions, such as

the disk simulator. In a realistic setting, there would likely be things like design meetings and code reviews that would allow for this, but for this project I had to accept not having control over these parts. While it should be simple to move on from this, I feel I did waste some time wishing I had control over the rest of the design or thinking about different ways around the current design given to me. Instead of this, I should have sooner accepted that all I could do was implement the cpmFS portion, and worked more in the path of that scope.

4. Don't Be Afraid To Start Over

One thing that did slow me down was my hesitancy to start over when work I had completed began to have friction with the rest of the system. For instance, my initial name parsing logic and rule enforcement began to encounter more and more problems as I expanded the cpmFS interface. Instead of continuing to make small adjustments constantly, I should have begun with the design I have now, where a centralized structure and utilities are given to any part of the project dealing with file names. This would have helped the rest of the project continue going smoother, even if it did require me to partially restart much of the project.

VII. Conclusion

In conclusion, I present my implementation for cpmFS, a simple file system. I discussed several design issues for the project and lessons I learned from my implementation.